

PRACTICAL EXPERIMENT – 7

Name: ANSHU

Roll No: 2520030440

Section: 08

Title

Process Synchronization and Child Process Monitoring Using waitpid().

Aim

To develop a C program that retrieves the PATH environment variable, parses the search directories, locates executable commands, verifies execute permissions, handles missing commands, and creates a child process to execute the command while the parent process monitors it using waitpid().

Theory

The PATH environment variable contains a list of directories in which Linux searches for executable commands. The getenv() function is used to retrieve the PATH variable.

The strtok() function is used to parse the PATH variable into individual directories. The access() system call is used to check whether an executable exists and whether the user has execute permission.

The fork() system call creates a child process. The child process can execute the located command using an exec() family system call. The parent process can use waitpid() to wait for a specific child process to finish and obtain its termination status.

If the requested command does not exist in any of the directories in PATH, the program displays an appropriate error message.

Algorithm

1. Start the program.
2. Retrieve the PATH environment variable using getenv().
3. Display the retrieved PATH.
4. Accept a Linux command from the user.
5. Parse the PATH variable into individual directories.
6. Search each directory for the requested executable.
7. Use access() to check whether the executable exists.
8. Verify execute permission using access(..., X_OK).
9. If the command is not found, display a missing-command message.
10. If the command is found, create a child process using fork().
11. In the child process, display the Child PID and execute the command using execl().

12. In the parent process, display the Parent PID.
13. Use `waitpid()` to wait for the child process.
14. Display the child process completion and exit status.
15. Stop the program.

Code

Paste the same code successfully executed:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_PATH 4096

int find_command(const char *command, char *result)
{
    char *path = getenv("PATH");
    char path_copy[MAX_PATH];
    char *dir;

    if (path == NULL)
        return 0;

    strcpy(path_copy, path);
    dir = strtok(path_copy, ":");

    while (dir != NULL)
    {
        printf("Checking directory: %s\n", dir);
        sprintf(result, "%s/%s", dir, command);

        if (access(result, F_OK) == 0)
        {
            printf("  File found: %s\n", result);

            if (access(result, X_OK) == 0)
            {
                printf("  Execute permission: YES\n");
                return 1;
            }
        }

        dir = strtok(NULL, ":");
    }

    return 0;
}

int main()
{
    char *path;
    char path_copy[MAX_PATH];
    char executable[MAX_PATH];
    char *dir;
    pid_t pid;
    int status;
    int found = 0;
```

```

printf("=====\n");
printf(" PROCESS SYNCHRONIZATION AND COMMAND TEST\n");
printf("=====\n");

printf("\n1. RETRIEVING PATH VARIABLE\n");
printf("-----\n");

path = getenv("PATH");

if (path == NULL)
{
    printf("PATH variable not found!\n");
    return 1;
}

printf("PATH = %s\n", path);

printf("\n2. SEARCHING PATH FOR COMMAND: ls\n");
printf("-----\n");

strcpy(path_copy, path);
dir = strtok(path_copy, ":");

while (dir != NULL)
{
    printf("Checking directory: %s\n", dir);
    sprintf(executable, "%s/%s", dir, "ls");

    if (access(executable, F_OK) == 0)
    {
        printf(" File found: %s\n", executable);

        if (access(executable, X_OK) == 0)
        {
            printf(" Execute permission: YES\n");
            found = 1;
            break;
        }
    }

    dir = strtok(NULL, ":");
}

if (!found)
{
    printf("Command 'ls' not found.\n");
    return 1;
}

printf("\nExecutable located successfully:\n");
printf("%s\n", executable);

printf("\n3. COMMAND RESOLUTION\n");
printf("-----\n");
printf("Command 'ls' is resolved successfully.\n");

```

```

printf("\n4. CREATING CHILD PROCESS\n");
printf("-----\n");

fflush(stdout);
pid = fork();

if (pid < 0)
{
    printf("Fork Failed!\n");
    return 1;
}
else if (pid == 0)
{
    printf("Child Process Started\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());
    printf("Executing command: ls\n\n");
    fflush(stdout);

    execl(executable, "ls", (char *)NULL);

    printf("Command execution failed!\n");
    exit(1);
}
else
{
    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("\n5. PROCESS SYNCHRONIZATION USING waitpid()\n");
    printf("-----\n");
    printf("Parent is waiting for child process...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child process completed normally.\n");
        printf("Child exit status: %d\n", WEXITSTATUS(status));
    }
    else
    {
        printf("Child process terminated abnormally.\n");
    }
}

printf("\n6. TESTING MISSING COMMAND\n");
printf("-----\n");

if (find_command("xyz123", executable))
    printf("Command 'xyz123' found.\n");
else
{
    printf("Command 'xyz123' not found.\n");
    printf("Missing command handled successfully.\n");
}

```

```
printf("\n=====\\n");
printf(" ALL TESTS COMPLETED SUCCESSFULLY\\n");
printf("=====\\n");

return 0;
}
```

Execution Screenshots

```
=====
PROCESS SYNCHRONIZATION AND COMMAND TEST
=====

1. RETRIEVING PATH VARIABLE
-----
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Python314/Scripts:/mnt/c/Python314/
:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common File
s/Oracle/Java/javapath:/mnt/c/windows/system32:/mnt/c/windows:/mnt/c/windows/System32/wbin:/mnt/c/windows/System32/WindowsPowerShell/v1.0:/mnt/c/windows/Sy
stem32/OpenSSH:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Program Files/dotnet:/mnt/c/Program Files/Java/
jdk-21/bin:/mnt/c/Users/mohit/AppData/Local/Microsoft/WindowsApps:/mnt/c/Program Files/Java/jdk-21/bin:/mnt/c/Users/mohit/AppData/Local/Programs/Microsoft V
S Code/bin:/mnt/c/Users/mohit/AppData/Local/Python/bin:/mnt/c/Users/mohit/AppData/Local/JetBrains/Toolbox/scripts:/mnt/c/Users/mohit/AppData/Roaming/npm:/mn
t/c/Users/mohit/AppData/Local/GitHubDesktop/bin:/mnt/c/Users/mohit/Documents/apache-maven-3.9.16-bin/apache-maven-3.9.16:/snap/bin

2. SEARCHING PATH FOR COMMAND: ls
-----
Checking directory: /usr/local/sbin
Checking directory: /usr/local/bin
Checking directory: /usr/sbin
Checking directory: /usr/bin
File found: /usr/bin/ls
Execute permission: YES

Executable located successfully:
/usr/bin/ls

3. COMMAND RESOLUTION
-----
Command 'ls' resolved successfully.

4. CREATING CHILD PROCESS
-----
Parent Process Started
Child Process Started
Parent PID: 1063
Child PID: 1064
```

Figure 1: PATH retrieval, executable search, permission verification and child process creation

```
5. PROCESS SYNCHRONIZATION USING waitpid()
Child PID: 1064
-----
Executing command: ls
Parent is waiting for child process...

os_command.c pipe pipe.c process process.c
Child process completed normally.
Child exit status: 0
```

Figure 2: Process synchronization using waitpid() and child exit status

```

6. TESTING MISSING COMMAND
-----
Checking directory: /usr/local/sbin
Checking directory: /usr/local/bin
Checking directory: /usr/sbin
Checking directory: /usr/bin
Checking directory: /sbin
Checking directory: /bin
Checking directory: /usr/games
Checking directory: /usr/local/games
Checking directory: /usr/lib/wsl/lib
Checking directory: /mnt/c/Python314/Scripts/
Checking directory: /mnt/c/Python314/
Checking directory: /mnt/c/Program Files/Common Files/Oracle/Java/javapath
Checking directory: /mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path
Checking directory: /mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath
Checking directory: /mnt/c/windows/system32
Checking directory: /mnt/c/windows
Checking directory: /mnt/c/windows/System32/Wbem
Checking directory: /mnt/c/windows/System32/WindowsPowerShell/v1.0/
Checking directory: /mnt/c/windows/System32/OpenSSH/
Checking directory: /mnt/c/Program Files/Git/cmd
Checking directory: /mnt/c/Program Files/
Checking directory: /mnt/c/ProgramData/chocolatey/bin
Checking directory: /mnt/c/Program Files/dotnet/
Checking directory: /mnt/c/Program Files/Java/jdk-21/bin
Checking directory: /mnt/c/Users/mohit/AppData/Local/Microsoft/WindowsApps
Checking directory: /mnt/c/Program Files/Java/jdk-21/bin
Checking directory: /mnt/c/Users/mohit/AppData/Local/Programs/Microsoft VS Code/bin
Checking directory: /mnt/c/Users/mohit/AppData/Local/Python/bin
Checking directory: /mnt/c/Users/mohit/AppData/Local/JetBrains/Toolbox/scripts
Checking directory: /mnt/c/Users/mohit/AppData/Roaming/npm
Checking directory: /mnt/c/Users/mohit/AppData/Local/GitHubDesktop/bin
Checking directory: /mnt/c/Users/mohit/Documents/apache-maven-3.9.16-bin/apache-maven-3.9.16
Checking directory: /snap/bin
Command 'xyz123' not found.
Missing command handled successfully.

=====
ALL TESTS COMPLETED SUCCESSFULLY
=====

```

Figure 3: Missing command handling and completion of all tests

Result

The program was executed successfully. The PATH variable was retrieved and parsed to search for the requested executable. The executable was located and its execute permission was verified. The command was executed through the child process, while the parent process monitored the child and waited for its completion using waitpid(). A missing command was also tested and handled successfully.